

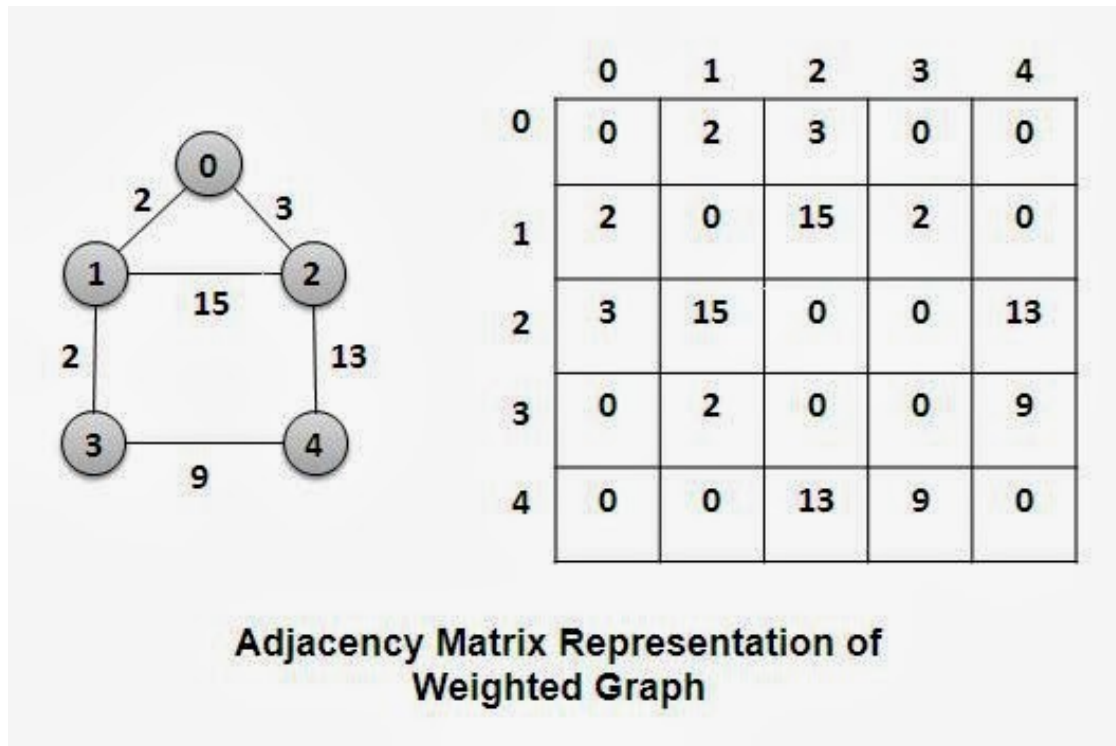
Práctica 6: Algoritmo de Dijkstra

El algoritmo de Dijkstra nos sirve para determinar el camino más corto, dado un vértice inicial, hacia el resto de los vértices en una gráfica con pesos en las aristas. Así, para nuestro nodo inicial A , tendremos un vector D que guardará al final del algoritmo las distancias desde A hasta el resto de los nodos.

1. **Inicializar.** Inicializamos todas las distancias en D con un valor infinito menos la de A , que se debe colocar en 0.
2. **Marcar el nodo permanente.** Se toma A como nodo permanente o definitivo.
3. **Identificar los nodos vecinos disponibles.** Se recorren todos los nodos adyacentes de A , excepto los nodos marcados como permanentes y llamamos a los nodos no marcados v_i .
4. **Retiquetado.** Para el nodo actual, se calcula la distancia temporal desde ese nodo a sus vecinos. La distancia temporal del vecino v_i es la distancia que actualmente tiene el nodo en el vector D más la distancia desde dicho nodo A (el nodo permanente) hasta el nodo v_i . Si la distancia tentativa es menor que la distancia almacenada en el vector, entonces se actualiza el vector con esta distancia tentativa.
5. **Actualizar nodo permanente.** Se toma como próximo nodo permanente el de menor valor en D y se regresa al paso 3, mientras existan nodos no marcados.

Para saber más pueden ver el siguiente video: <https://www.youtube.com/watch?v=LLx0QVMZVkk>

Observaciones.



- **MD.** Una matriz de costos de las aristas del grafo, donde en $MD[u,v]$ se almacena el costo de la arista entre u y v
- **ED.** Conjunto que contendrá los vértices para los cuales ya se tiene determinado el camino mínimo.
- **D.** Un arreglo unidimensional tal que $D[v]$ es el costo del camino mínimo del vértice origen al vértice v .
- **O.** Un arreglo unidimensional tal que $P[v]$ es el vértice predecesor de v en el camino mínimo que se tiene construido

Ejercicio 1

Programa una función que reciba la matriz de pesos de una gráfica y el nodo inicial y que aplique el algoritmo de Dijkstra. Tu función debe regresar una lista con las distancias de las rutas y el origen de la arista con la que terminó la ruta

```
import numpy as np
import matplotlib.pyplot as plt
```

```
MD = np.zeros((4,4))
MD[0,1] = 9
MD[3,2] = 2
MD[0,3] = 6
MD[1,3] = 1
MD[2,1] = 3
MD
```

```
array([[0., 9., 0., 6.],  
       [0., 0., 0., 1.],  
       [0., 3., 0., 0.],  
       [0., 0., 2., 0.]])
```

Solución

Ejercicio 2

Ahora, usando las listas generadas por tu función del algoritmo de Dijkstra, programa una función que encuentre el camino óptimo entre dos vértices

Solución

Ejercicio 3

Prueba tus funciones con las siguientes matrices de pesos, empezando siempre en el nodo 0.

Nota : Donde encuentres un cero quiere decir que no existe una arista entre dichos vertices

Gráfica 1

La matriz de pesos está dada por:

$$\begin{bmatrix} 0 & 3 & 0 & 2 & 0 & 0 & 0 & 0 \\ 3 & 0 & 1 & 0 & 4 & 0 & 0 & 0 \\ 0 & 1 & 0 & 3 & 0 & 2 & 2 & 0 \\ 2 & 0 & 3 & 0 & 0 & 0 & 4 & 0 \\ 0 & 4 & 0 & 0 & 0 & 0 & 0 & 6 \\ 0 & 0 & 2 & 0 & 0 & 0 & 3 & 4 \\ 0 & 0 & 2 & 4 & 0 & 3 & 0 & 5 \\ 0 & 0 & 0 & 0 & 6 & 4 & 5 & 0 \end{bmatrix}$$

La definimos en python para ahorrarles trabajo:

```
M1 = np.zeros((8,8))
```

```
M1[0,1] = M1[1,0] = 3
M1[1,2] = M1[2,1] = 1
M1[0,3] = M1[3,0] = 2
M1[3,2] = M1[2,3] = 3
M1[1,4] = M1[4,1] = 4
M1[2,5] = M1[5,2] = 2
M1[2,6] = M1[6,2] = 2
M1[3,6] = M1[6,3] = 4
M1[4,7] = M1[7,4] = 6
M1[5,7] = M1[7,5] = 4
M1[5,6] = M1[6,5] = 3
M1[6,7] = M1[7,6] = 5
```

En esta celda prueba tus funciones:

Solución:

Gráfica 2

Ahora tenemos una gráfica dirigida

La matriz de pesos está dada por:

$$\begin{bmatrix} 0 & 9 & 0 & 6 \\ 0 & 0 & 0 & 1 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & 2 & 0 \end{bmatrix}$$

La definimos en python para ahorrarles trabajo:

```
M2 = np.zeros((4,4))
```

```
M2[0,1] = 9
```

```
M2[3,2] = 2
```

```
M2[0,3] = 6
```

```
M2[1,3] = 1
```

```
M2[2,1] = 3
```

En esta celda prueba tus funciones:

Solución

Gráfica 3

Otra una gráfica dirigida
La matriz de pesos está dada por:

$$\begin{bmatrix} 0 & 4 & 8 & 16 \\ 0 & 0 & 5 & 11 \\ 0 & 0 & 0 & 6 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

La definimos en python para ahorrarles trabajo:

```
M3 = np.zeros((4,4))
```

```
M3[0,1] = 4  
M3[0,2] = 8  
M3[0,3] = 16  
M3[1,2] = 5  
M3[1,3] = 11  
M3[2,3] = 6
```

En esta celda prueba tus funciones:

Solución

Ejercicio 4:

Encuentra la distancia mínima para el siguiente ejemplo , y organice el diagrama para tenerlo en Python

